1 DAY

VOICE & SPEECH AI

Build a Voice Assistant

Guided one-day lab • beginner to intermediate

Students build a complete spoken-conversation loop: they talk to their laptop and it talks back. Three models are chained together -- Whisper for speech recognition, a language model for the reply, and a speech synthesiser for the voice. Each individual stage is short. The engineering that makes it feel like a conversation is the lesson.

At a glance	
Duration	~6 hours including breaks
Level	Beginner to intermediate. Python and basic NumPy assumed. No audio or speech background needed.
Hardware	Any laptop. CPU is sufficient; a GPU speeds up Whisper but is not required.
Cost	Zero. Whisper and the local LLM path are free and offline.
Deliverable	Completed notebook, a saved audio demo, and four short written answers.
Assessment	100 points across 8 criteria, plus up to 10 bonus.

- **Handling the absence of input.** Whisper hallucinates confident text from silence. Students meet this failure directly and build a confidence filter against it.
- **Choosing a model with evidence.** Six model sizes, a real speed/accuracy trade-off, and a decision that has to be justified with measured numbers.
- **Writing for the ear.** Every habit an LLM has -- markdown, bullet lists, long structured answers -- is wrong when the output is spoken. This surprises students, and fixing it is mostly prompt design.
- **Profiling a pipeline.** Three stages, one latency budget. Students measure before they optimise, which is the habit worth transferring to everything else.

Learning outcomes

On completing this lab, a student can:

- Transcribe audio with Whisper and interpret its segment-level output, including timestamps, `avg_logprob` and `no_speech_prob`.
- Explain what a log-Mel spectrogram is and why Whisper's encoder consumes one rather than a raw waveform.
- Justify a Whisper model-size choice from their own latency and accuracy measurements.
- Write a system prompt tuned for spoken rather than written output.
- Maintain multi-turn conversation memory within a bounded history.
- Profile a multi-stage inference pipeline and identify the dominant cost.

Prerequisites

Assumed	Explicitly NOT assumed
Python: functions, classes, dictionaries, list comprehensions	Any signal-processing or audio background
NumPy array basics and slicing	Prior use of Whisper, or of any speech model
Comfort running cells in Jupyter or Colab	Transformer internals (the lab explains what it needs)
Having called an API or a local model at least once	A GPU, an API key, or a paid account

Setup

1 DAY

r is ffmpeg.

```
pip install openai-whisper sounddevice scipy matplotlib numpy requests gTTS

# ffmpeg (system package, pick your platform)
Ubuntu / Colab : apt-get install -y ffmpeg
macOS : brew install ffmpeg
Windows : winget install ffmpeg

# LLM backend -- recommended, free and fully offline
# install Ollama from ollama.com, then:
ollama pull llama3.2
```

Everything degrades gracefully

The notebook is written so no missing component blocks progress. No microphone? Test clips are synthesised with TTS. No LLM? A scripted keyword responder keeps the pipeline complete. No TTS engine? Replies are printed. A student with nothing but Whisper installed still finishes the lab.

Common setup failures

Symptom	Cause and fix
`FileNotFoundError: ffmpeg`	ffmpeg is not on PATH. Install it per the box above, then **restart the kernel** -- PATH is read at process start.
Model download stalls or fails	Weights come from a CDN on first use and are cached in `~/.cache/whisper`. On a restricted network, download once and distribute the cache folder to students on a USB stick.
`sounddevice` finds no input device	Expected on Colab and on most VMs. Use the synthesised-clip path (option A) instead; it needs no microphone.
Ollama connection refused	The server is not running. Start it with `ollama serve`, and confirm a model is pulled with `ollama list`.
Transcription is extremely slow	Almost always CPU with a large model. Drop to `tiny.en`. Note that every clip is padded to 30 s, so a 2 s clip costs the same as a 25 s one.
pyttsx3 silent on Linux	It needs espeak: `apt-get install espeak`. Or use gTTS, which needs internet but no system package.

Hour-by-hour plan

behind 1 DAY

Part	Topic	What students produce	Time
1	Getting audio in	A 16 kHz clip from mic, file, or synthesis, plus a plotted waveform.	30 min
2	**Whisper: the ears**	Transcription, segment inspection, a Mel spectrogram, a measured size/latency benchmark, and a silence-rejection wrapper.	75 min
3	The brain	An LLM call with a voice-first system prompt and a trimmed conversation memory.	60 min
4	The mouth	A working `speak()` and a text sanitiser that strips markdown before synthesis.	40 min
5	**Assembling the loop**	A `VoiceAssistant` class, a three-turn conversation, and a latency breakdown chart.	60 min
6	Making it feel real	Energy-based endpointing, a record-until-silence function, and a wake word.	45 min
7	Stretch and submission	One extension attempted; four written answers.	30 min

Suggested break points

After Part 2 (a natural high point -- their machine just understood them) and after Part 4. Part 5 should be run in one uninterrupted block, because that is where the three pieces click together and the momentum matters.

> **Teaching note: let the silence failure happen**
> In Part 2 students transcribe pure silence and Whisper often returns confident text, commonly a subtitle-style artefact from its training data. Resist warning them first. The surprise is what makes the confidence-filtering lesson stick, and it is a rare chance to show a real model failing in a way that has nothing to do with a bug in their code.

Deliverables

1 DAY

Written answers (2-3 sentences each)

- Why does Whisper convert audio to a log-Mel spectrogram instead of feeding the raw waveform to the Transformer?
- Which Whisper model size did you choose for the live loop, and what measurement justifies it?
- Which pipeline stage dominated your latency, and what single change would help most?
- Name one specific way your assistant fails, and how you would fix it.

Assessment rubric

#	Criterion	Full marks means	Pts
1	**Speech in**	Whisper transcribes their audio; segment fields are printed and correctly interpreted in comments or discussion.	**15**
2	**Robustness**	`robust_transcribe` returns an empty string for silence and correct text for real speech; the threshold choice is explained.	**10**
3	**Evidence**	The model-size benchmark is filled in with their own measured numbers, not the table from the notes.	**15**
4	**The brain**	An LLM backend is wired up and the system prompt is genuinely voice-first -- short, no markdown, numbers as words.	**15**
5	**Memory**	A three-turn conversation where the third turn cannot be answered without context from the first.	**10**
6	**Voice out**	TTS produces audible output and `clean_for_speech` handles markdown, symbols and URLs.	**10**
7	**Integration**	The full loop runs end to end and the latency breakdown chart is produced from their own timings.	**15**
8	**Reflection**	All four written answers present, specific, and consistent with what they actually measured.	**10**
	Total		**100**
+	**Stretch challenge**	One extension from Part 7 attempted, with an honest note on whether it worked. Partial credit for a well-described failure.	**+10**

Marking guidance

Criterion 3 is the one to hold the line on. Students frequently paste the reference table instead of running the benchmark, because the reference numbers look more authoritative than their own noisy laptop measurements. Their noisy numbers are the point -- award marks for measuring and interpreting, not for matching a published figure.

Whisper reference

1 DAY

Model	Params	VRAM	Rel. speed	Use it when
`base`	74 M	~1 GB	~7x	**Default for this lab.** Good balance for live use.
`small`	244 M	~2 GB	~4x	Accents or background noise start to hurt.
`medium`	769 M	~5 GB	~2x	Offline batch transcription with a GPU.
`large-v3`	1550 M	~10 GB	1x	Maximum accuracy; not for real-time on CPU.
`turbo`	809 M	~6 GB	~8x	Near-large accuracy, fast. Transcription only, no translation.

Facts worth having at hand

- Trained on **680,000 hours** of weakly supervised multilingual audio across 98 languages -- roughly 100x the labelled data of the academic benchmarks of its time.
- Audio is padded or trimmed to exactly **30 seconds**, then turned into an **80 x 3000** log-Mel spectrogram (128 mel bins for `large-v3`).
- Because of that fixed window, a 2-second clip costs the same compute as a 25-second one.
- It is an **encoder-decoder Transformer**. The task -- transcribe, translate, detect language, emit timestamps -- is selected by **special tokens fed to the decoder**, not by different weights.
- `no_speech_prob` is the model's own estimate that a segment contains no speech. It is the standard defence against hallucinated silence.

Latency targets

Useful to put on the board during Part 5. Students consistently underestimate how quickly a delay becomes uncomfortable.

Total turn latency	How it feels
under 0.5 s	Indistinguishable from talking to a person.
0.5 - 1.5 s	Natural. This is the target.
1.5 - 3 s	Noticeably slow, but usable.
over 3 s	The user starts talking again or repeats themselves. The illusion of conversation breaks.

Where to go next

- **faster-whisper** -- a CTranslate2 reimplementation, typically 3-4x quicker for identical weights and lower memory. The right choice for anything real-time.
- **Silero VAD** or **webrtcvad** -- proper voice activity detection, for when RMS energy stops coping with background noise.
- **openWakeWord** or **Porcupine** -- tiny always-on wake-word models, so the heavy pipeline only wakes on demand.

3.

Whisper

1 DAY

The natural follow-on project

Streaming. Have the LLM emit tokens continuously, buffer to sentence boundaries, and start synthesising sentence one while sentence two is still being written. It is the largest perceived-latency win available and it introduces async pipelining on a problem where students can hear the improvement immediately.